NHÓM 5 THÀNH VIÊN · FLUTTER · FIREBASE · SUPABASE

Hướng dẫn triển khai dự án Quản lý Tài liệu

Từ clone mã nguồn đến chạy ứng dụng trên Android và Web

Dành cho thành viên nhóm: Minh · Đạt · Hạnh · Quân · Đăng

Repository	github.com/Minh-lab/document-management
Phiên bản	1.0 — 20/10/2026
Công nghệ	Flutter (Dart) · Firebase Auth/Messaging · Supabase Database/Storage
Nền tảng	Android · Web (Chrome)

Mục lục

1 Tổng quan kiến trúc 3

2 Chuẩn bị môi trường 4

3 Clone mã nguồn và cấu trúc thư mục 5

4 Thiết lập Supabase (Database & Storage) 6

5 Thiết lập Firebase (Authentication & Messaging) 8

6 Cấu hình ứng dụng và chạy 10

7 Checklist kiểm tra sau khi thiết lập 11

8 Lỗi thường gặp và cách xử lý 12

9 Quy trình Git làm việc nhóm 13

10 Bảo mật và lưu ý quan trọng 14

> **Đường nhanh:** nếu bạn chỉ cần chạy app để code giao diện, hãy dùng chung Firebase/Supabase project của nhóm (khóa đã có trong repo) — chỉ cần làm mục 2, 3, 6 rồi sang mục 7. Chỉ làm mục 4 và 5 khi bạn cần project riêng hoặc được giao phụ trách backend.

1. Tổng quan kiến trúc

Ứng dụng dùng **hai dịch vụ đám mây song song**, phân chia trách nhiệm rõ ràng:

Thành phần	Dịch vụ	Nhiệm vụ
Xác thực	Firebase Authentication	Đăng nhập ẩn danh (anonymous) và Google Sign-In; sinh UID duy nhất cho mỗi người dùng
Thông báo	Firebase Cloud Messaging	Đẩy thông báo khi có yêu cầu mượn tài liệu (tính năng mở rộng)
Cơ sở dữ liệu	Supabase (PostgreSQL)	Bảng `users`, `documents`, `categories`, `permissions`
Lưu trữ tệp	Supabase Storage	Bucket `documents`, đường dẫn `users/{uid}/documents/{docId}/{file}`

Luồng khởi động ứng dụng

```
main()
 ├─ Firebase.initializeApp(options: DefaultFirebaseOptions)
 ├─ Supabase.initialize(url, publishableKey)
 ├─ initializeServices()
 │    ├─ Firebase Auth: đăng nhập ẩn danh nếu chưa có user
 │    ├─ Upsert user vào bảng users (Supabase)
 │    └─ Seed danh mục mặc định
 └─ runApp(HomePage)
```

> **Vì sao cần cả hai?**
>
> Firebase lo phần *xác thực và thông báo* (miễn phí, SDK ổn định trên mobile/web); Supabase lo phần *dữ liệu quan hệ và tệp* (PostgreSQL + Storage + truy vấn realtime). Hai bên liên kết bằng **Firebase UID** được lưu trong cột `user_id`.

Thông tin project của nhóm

Hạng mục	Giá trị
Firebase project	`document-management-f7883`
Android applicationId	`com.example.code`
Supabase URL	`https://lwebnjcszbldlqwjrcan.supabase.co`
Storage bucket	`documents`

2. Chuẩn bị môi trường

2.1. Phần mềm bắt buộc

Công cụ	Phiên bản	Cách kiểm tra
Git	≥ 2.30	`git --version`
Flutter SDK	3.x (Dart ≥ 3.13)	`flutter --version`
JDK	17 (cho Android)	`java -version`
Android SDK / Android Studio	API 34+	`flutter doctor`
Chrome	Bản mới nhất	Dùng để chạy Flutter Web
VS Code / Android Studio	—	Cài extension Flutter + Dart

```
# Kiểm tra tổng thể, phải hết dấu "X" ở mục Flutter, Android toolchain, Chrome
flutter doctor -v
```

2.2. Công cụ hỗ trợ thiết lập (chỉ cần khi làm mục 4, 5)

```
# Firebase CLI + FlutterFire CLI
npm install -g firebase-tools
dart pub global activate flutterfire_cli

# Supabase CLI
npm install -g supabase

# Đăng nhập
firebase login
supabase login
```

> **Windows**
>
> Nếu `flutterfire` không nhận lệnh, thêm thư mục sau vào `PATH`: `%LOCALAPPDATA%\Pub\Cache\bin`. Trên macOS/Linux: `$HOME/.pub-cache/bin`.

3. Clone mã nguồn và cấu trúc thư mục

```
git clone https://github.com/Minh-lab/document-management.git
cd document-management
```

Repo gồm hai phần: mã nguồn ứng dụng và tài liệu.

```
document-management/
├── code/                  # Ứng dụng Flutter
│   ├── lib/
│   │   ├── main.dart              # Khởi tạo Firebase + Supabase
│   │   ├── functions.dart         # Nghiệp vụ dùng chung
│   │   ├── config/
│   │   │   ├── supabase_config.dart   # URL + publishable key
│   │   │   └── firebase_config.dart   # Placeholder
│   │   ├── firebase_options.dart      # FlutterFire sinh ra
│   │   ├── pages/                 # Màn hình (UI)
│   │   ├── widgets/               # Widget dùng lại
│   │   ├── services/              # Supabase / Storage
│   │   └── struct/                # Model dữ liệu
│   ├── android/app/google-services.json
│   ├── supabase/functions/verify-firebase-token/
│   ├── supabase_schema.sql
│   ├── supabase_rls_policies.sql
│   ├── supabase_storage_policies.sql
│   └── pubspec.yaml
└── docs/                  # Đặc tả, báo cáo, kế hoạch phân công
```

Cài dependencies

```
cd code
flutter pub get
```

> **Quan trọng**
>
> Mọi lệnh `flutter`, `dart`, `firebase` trong tài liệu này được chạy **bên trong thư mục code/**, không phải thư mục gốc repo.

4. Thiết lập Supabase (Database & Storage)

> **Trường hợp thường gặp: dùng project chung của nhóm**
>
> Project `lwebnjcszbldlqwjrcan` đã được cấu hình sẵn và khóa đã nằm trong `code/lib/config/supabase_config.dart`. Bạn **không cần làm mục 4.1–4.6**, chỉ cần xác nhận app kết nối được (mục 4.7). Liên hệ Minh để được thêm vào organization nếu cần xem dashboard.

4.1. Tạo project (chỉ khi làm project riêng)

1. Mở supabase.com/dashboard → **New project**.
2. Đặt tên (ví dụ `doc-manager-db`), chọn region **Southeast Asia (Singapore)**.
3. Đặt database password mạnh và lưu lại; chờ khoảng 2 phút để project khởi tạo.

4.2. Chạy schema theo đúng thứ tự

Vào **SQL Editor → New query**, dán và chạy lần lượt đúng thứ tự sau (mỗi file một lần chạy):

#	Tệp	Tác dụng
1	`supabase_schema.sql`	Tạo bảng `categories`, `documents`, index, trigger `updated_at`, seed danh mục, bật realtime
2	`supabase_rls_policies.sql`	Tạo bảng `users`, `permissions` và các policy RLS
3	`supabase_storage_policies.sql`	Cấp quyền CRUD bucket `documents` cho người dùng đã đăng nhập

> **Thứ tự quan trọng**
>
> File 3 tham chiếu bucket và policy storage, phải chạy **sau** khi đã tạo bucket ở mục 4.3. Nếu chạy sai thứ tự, chỉ cần chạy lại file đó — các lệnh đều dùng `IF NOT EXISTS` / `DROP POLICY IF EXISTS`.

4.3. Tạo Storage bucket

1. Vào **Storage → New bucket**.
2. Tên bucket: `documents` (chính xác, phân biệt chữ thường).
3. Bật **Public bucket** để tệp có thể mở bằng URL công khai (phù hợp đồ án).
4. Sau khi tạo, chạy `supabase_storage_policies.sql` ở SQL Editor.

4.4. Bật Realtime

File schema đã thêm bảng vào publication. Kiểm tra tại **Database → Replication**: `documents` và `categories` phải ở trạng thái bật. Nếu chưa, bật thủ công.

4.5. Lấy khóa API

Vào **Settings → API**:

- **Project URL** — dạng `https://<project-ref>.supabase.co`
- **anon / publishable key** — khóa công khai dùng trong app
- **service_role key** — **tuyệt đối không đưa vào app**

4.6. Cập nhật cấu hình trong code

Mở `code/lib/config/supabase_config.dart` và thay hai giá trị:

```
class SupabaseConfig {
  static const String supabaseUrl = 'https://<project-ref>.supabase.co';
  static const String supabaseAnonKey = '<anon-publishable-key>';
  // ...
}
```

4.7. (Tùy chọn) Deploy Edge Function đồng bộ Firebase → Supabase

Hàm `verify-firebase-token` xác thực Firebase ID token và cấp JWT cho Supabase, giúp RLS nhận đúng `auth.uid()`. Nếu chưa deploy, app vẫn chạy nhưng RLS chỉ hoạt động ở mức anon.

```
# Gắn project (chạy ở thư mục code/)
supabase link --project-ref <project-ref>

# Nạp biến môi trường cho function
supabase secrets set FIREBASE_PROJECT_ID=document-management-f7883
supabase secrets set SUPABASE_JWT_SECRET=<jwt-secret-trong-Settings-API>

# Deploy (bỏ xác thực JWT vì function tự xác thực Firebase token)
supabase functions deploy verify-firebase-token --no-verify-jwt
```

> **Kiểm tra nhanh bằng curl**
>
> `curl -i https://<project-ref>.supabase.co/functions/v1/verify-firebase-token` — trả về `401` kèm JSON là function đã sống (thiếu token là đúng).

5. Thiết lập Firebase (Authentication & Messaging)

> **Dùng project chung**
>
> Nếu bạn dùng project `document-management-f7883` của nhóm, các tệp `lib/firebase_options.dart` và `android/app/google-services.json` đã có trong repo — **bỏ qua mục 5.1–5.4**. Chỉ cần đảm bảo bạn được thêm làm thành viên Firebase project (liên hệ Minh) khi cần xem console.

5.1. Tạo Firebase project

1. Mở console.firebase.google.com → **Add project**.
2. Đặt tên (ví dụ `document-management`), có thể tắt Google Analytics cho gọn.
3. Chờ khởi tạo xong → vào project.

5.2. Đăng ký ứng dụng Android

1. **Project settings → Your apps → Android**.
2. Package name: `com.example.code` — phải khớp `applicationId` trong `android/app/build.gradle.kts`, nếu không app sẽ crash khi khởi động.
3. Tải `google-services.json` và đặt vào `code/android/app/google-services.json`.

5.3. Đăng ký ứng dụng Web (để chạy `flutter run -d chrome`)

Trong **Project settings → Your apps → Web (</>)**, đặt nickname bất kỳ rồi copy đoạn `firebaseConfig`. Bước 5.4 sẽ tự sinh cấu hình này, không cần dán tay vào `index.html`.

5.4. Sinh cấu hình bằng FlutterFire CLI

```
# Chạy trong thư mục code/
flutterfire configure --project=<firebase-project-id>

# Chọn các nền tảng: android, web (có thể thêm ios, windows, linux)
# Kết quả: ghi đè lib/firebase_options.dart + tải google-services.json
```

> **Package name không khớp**
>
> Nếu đặt applicationId khác `com.example.code`, phải sửa `namespace` và `applicationId` trong `android/app/build.gradle.kts`, đổi tên thư mục `android/app/src/main/kotlin/com/example/code/`, rồi đăng ký lại app trên Firebase. Cách nhanh nhất là giữ nguyên `com.example.code`.

5.5. Bật phương thức đăng nhập

Provider	Thiết lập
Anonymous	**Authentication → Sign-in method → Anonymous → Enable**. Bắt buộc, vì app tự đăng nhập ẩn danh khi chưa có tài khoản.
Google	Bật Google → chọn support email. Trên Android cần thêm **SHA-1** của máy vào Project settings (lấy bằng `cd android && ./gradlew signingReport`).

5.6. Quy tắc Storage (chỉ khi dùng Firebase Storage)

Ứng dụng hiện lưu tệp trên **Supabase Storage**; Firebase Storage chỉ còn là mã cũ. Nếu vẫn muốn dùng, vào **Storage → Rules**:

```
rules_version = '2';
service firebase.storage {
  match /b/{bucket}/o {
    match /users/{userId}/documents/{docId}/{fileName} {
      allow read: if true;
      allow write: if request.auth != null && request.auth.uid == userId;
    }
  }
}
```

5.7. Bật Cloud Messaging (cho tính năng thông báo)

1. Android: thêm dependency `firebase_messaging` vào `pubspec.yaml` (nếu chưa có).
2. Android 13+: xin quyền `POST_NOTIFICATIONS` trong `AndroidManifest.xml`.
3. Web: tạo cặp khóa Web Push tại **Project settings → Cloud Messaging → Web configuration**, khai báo `firebase-messaging-sw.js` trong `web/`.
4. Lưu FCM token vào bảng `device_tokens` trên Supabase để Edge Function gửi thông báo.

> **Không commit khóa bí mật**
>
> `google-services.json` và `firebase_options.dart` chỉ chứa định danh công khai của app (apiKey dạng `AIza...` không phải bí mật), nhưng **service account JSON**, `service_role` key và JWT secret thì tuyệt đối không được đưa lên Git.

6. Cấu hình ứng dụng và chạy

6.1. Kiểm tra hai tệp cấu hình

Tệp	Nội dung cần đúng
`lib/config/supabase_config.dart`	`supabaseUrl`, `supabaseAnonKey`
`lib/firebase_options.dart`	`projectId`, `apiKey`, `appId` của Firebase

6.2. Chạy ở chế độ debug

```
cd code
flutter pub get

flutter devices                 # xem danh sách thiết bị
flutter run -d chrome           # Web
flutter run -d emulator-5554    # Android (đổi theo id thực tế)
```

> **Web lần đầu**
>
> Nếu trình duyệt chặn popup khi đăng nhập Google, cho phép popup cho `localhost`. Với đăng nhập ẩn danh thì không cần bước này.

6.3. Build bản phát hành

```
# Android APK
flutter build apk --release
# Kết quả: build/app/outputs/flutter-apk/app-release.apk

# Web
flutter build web --release
# Kết quả: build/web/ — có thể deploy lên Firebase Hosting, Vercel, Netlify
```

6.4. Lệnh kiểm tra chất lượng trước khi mở PR

```
flutter analyze
flutter test
dart format --set-exit-if-changed lib test
```

7. Checklist kiểm tra sau khi thiết lập

Chạy app và đánh dấu từng mục. Nếu mục nào sai, xem bảng lỗi ở mục 8.

Khởi động và xác thực

- [ ] App mở không có màn hình trắng, không lỗi đỏ trên console
- [ ] Log có dòng xác nhận Firebase và Supabase đã khởi tạo
- [ ] Người dùng được tạo trong Firebase Authentication (tab Users)
- [ ] Bản ghi tương ứng xuất hiện trong bảng `users` trên Supabase

Dữ liệu và tệp

- [ ] Danh mục mặc định hiển thị (Bài giảng, Bài tập, Tham khảo)
- [ ] Thêm tài liệu không kèm tệp thành công
- [ ] Thêm tài liệu có tệp: tệp xuất hiện trong bucket `documents`
- [ ] Mở tệp từ danh sách tải được nội dung
- [ ] Sửa tài liệu: `updated_at` thay đổi
- [ ] Xóa tài liệu: bản ghi và tệp đều bị xóa

Chia sẻ và realtime

- [ ] Chia sẻ tài liệu cho người dùng khác với quyền viewer/editor
- [ ] Người được chia sẻ thấy tài liệu trong mục "Được chia sẻ"
- [ ] Mở hai tab: thêm/sửa ở tab 1, tab 2 tự cập nhật
- [ ] Đăng xuất và đăng nhập lại vẫn thấy dữ liệu cũ

Đa nền tảng

- [ ] Chạy được trên Chrome và trên Android (hoặc emulator)
- [ ] Thu nhỏ cửa sổ trình duyệt: layout không tràn, không lỗi overflow
- [ ] Bàn phím ảo trên mobile không che ô nhập liệu

8. Lỗi thường gặp và cách xử lý

Lỗi / hiện tượng	Nguyên nhân	Cách xử lý
`Failed to load firebase_options.dart` / `FirebaseAppNotInitialized`	Chưa chạy `flutterfire configure` hoặc file bị ghi đè	Chạy lại `flutterfire configure`, đảm bảo `main.dart` gọi `Firebase.initializeApp(options: DefaultFirebaseOptions.currentPlatform)`
App Android crash ngay khi mở	Thiếu `google-services.json` hoặc package name không khớp	Đặt file đúng `android/app/`; kiểm tra `applicationId` = package đã đăng ký
`Supabase not initialized`	Chưa gọi `Supabase.initialize()` hoặc sai URL/key	Kiểm tra `main.dart` và `supabase_config.dart`
`new row violates row-level security policy`	RLS bật nhưng chưa có policy phù hợp, hoặc `auth.uid()` là NULL	Chạy `supabase_rls_policies.sql`; kiểm tra người dùng đã đăng nhập; xem lại policy của bảng liên quan
`Bucket not found`	Chưa tạo bucket hoặc sai tên	Tạo bucket tên chính xác `documents`, chạy lại file storage policies
`Permission denied` khi upload	Bucket private hoặc thiếu policy INSERT	Bật public bucket hoặc chạy `supabase_storage_policies.sql`
CORS error trên Web	Domain chưa được cho phép / chưa cấu hình CORS cho storage	Supabase: thêm `localhost` vào URL Configuration; Firebase: cấu hình CORS bucket
Realtime không cập nhật	Bảng chưa được thêm vào publication	Database → Replication → bật cho `documents`, `categories`
Đăng nhập Google thất bại trên Android	Thiếu SHA-1 / SHA-256	`cd android && ./gradlew signingReport`, thêm fingerprint vào Firebase rồi tải lại `google-services.json`
`Gradle build failed` / JDK mismatch	Sai phiên bản JDK	Dùng JDK 17: `flutter config --jdk-dir=<đường-dẫn-jdk-17>`
Đã pull code mới nhưng vẫn lỗi cũ	Cache build	`flutter clean && flutter pub get`, xóa `build/` và `.dart_tool/` nếu cần

> **Thứ tự debug nên theo**
>
> 1) Console Flutter → 2) Log Firebase (Authentication / Storage) → 3) Log Supabase (API / Database / Realtime) → 4) Tab Network trong DevTools khi chạy Web.

9. Quy trình Git làm việc nhóm

> **Không push trực tiếp lên main**
>
> Nhánh `main` là bản chạy được. Mọi thay đổi đi qua nhánh riêng và Pull Request.

9.1. Tạo nhánh và làm việc

```
git checkout main
git pull origin main
git checkout -b feature/ten-tinh-nang

# ... code ...

git add .
git commit -m "feat: mô tả ngắn gọn thay đổi"
git push -u origin feature/ten-tinh-nang
```

9.2. Trước khi mở Pull Request

```
git fetch origin
git rebase origin/main        # giải quyết conflict ngay tại máy

flutter analyze
flutter test
flutter run -d chrome         # kiểm tra giao diện
```

9.3. Quy ước đặt tên

Loại	Mẫu	Ví dụ
Nhánh tính năng	`feature/<ten>-<viec>`	`feature/hanh-tag-location`
Nhánh sửa lỗi	`fix/<mo-ta>`	`fix/login-google-android`
Commit	`feat|fix|docs|test|refactor|chore: ...`	`feat: thêm lọc theo tag`

9.4. Điều kiện để PR được merge

- [ ] Ít nhất 1 người review và approve
- [ ] `flutter analyze` không có lỗi
- [ ] `flutter test` pass toàn bộ, kể cả `architecture_test.dart`
- [ ] Không commit tệp build, `.dart_tool/`, `.idea/` hay khóa bí mật
- [ ] Có ảnh chụp màn hình nếu thay đổi giao diện

10. Bảo mật và lưu ý quan trọng

Loại khóa	Được commit?	Ghi chú
Supabase `anon` / `publishable`	Có	Thiết kế để nằm trong app; bảo vệ bằng RLS
Firebase `apiKey`, `appId`	Có	Định danh công khai của app, không phải bí mật
Supabase `service_role`	Không	Bỏ qua toàn bộ RLS — chỉ dùng trong Edge Function/script server
Supabase JWT secret	Không	Chỉ đặt bằng `supabase secrets set`
Firebase service account JSON	Không	Dùng cho Admin SDK ở server

Nguyên tắc RLS

- Bật RLS cho **mọi bảng** chứa dữ liệu người dùng.
- Policy đọc phải kiểm tra cả chủ sở hữu và người được chia sẻ: `user_id = auth.uid()::text OR EXISTS (SELECT 1 FROM permissions ...)`.
- Chỉ chủ sở hữu mới được `DELETE` tài liệu; người có quyền editor chỉ được `UPDATE`.
- Luôn kiểm thử bằng **hai tài khoản**: A không được đọc tài liệu riêng của B.

> **Hiện trạng cần lưu ý**
>
> Bản hiện tại dùng Firebase Auth nhưng chưa có custom JWT cho Supabase, nên `auth.uid()` có thể là `NULL` và một số policy đang được nới lỏng để app chạy được. Khi Edge Function `verify-firebase-token` được deploy và app set session thành công, hãy siết lại policy theo UID thật. Đây là việc của Minh (hạng mục #1) và cần review chéo.

Quy ước khi thêm bảng mới

```
ALTER TABLE ten_bang ENABLE ROW LEVEL SECURITY;

CREATE POLICY "chu so huu doc"
  ON ten_bang FOR SELECT
  USING (user_id = auth.uid()::text);
```

Hỗ trợ

Khi gặp vấn đề, gửi vào nhóm kèm: **lệnh đã chạy**, **log đầy đủ**, **ảnh chụp màn hình** và cho biết đang chạy **Web hay Android** — ba thông tin này giúp xử lý nhanh hơn nhiều so với mô tả chung chung.
